

Lab 4 Web Crawling & HTTP API Integration - Tasks

Duration: 2.5 hours | **Points:** 100

Lab Overview

Objective: Learn web scraping, HTTP APIs, and data collection techniques in Go to build robust data collection systems.

What You'll Build: Web crawlers and API clients that collect, parse, and store structured data from websites and APIs - working with movie databases.

Why This Matters for Your Project:

The skills you learn here apply directly to your MangaHub project:

- Scraping websites for structured information (movies → manga)
- Integrating with external APIs (TMDB → MangaDex)
- Parse HTML and extract data from multiple sources
- Store collected data in JSON format
- Handle rate limiting and error management
- Build multi-source data aggregators

Key Transferable Skills:

-  REST API integration patterns
-  Web scraping techniques
-  Data normalization from multiple sources
-  JSON database management
-  Error handling and rate limiting

Setup (5 minutes)

Create New Project

```
mkdir web-crawler-lab
cd web-crawler-lab
go mod init crawler-lab
```

Install Required Packages

```
# HTML parsing library
go get golang.org/x/net/html

# HTTP client with better features (optional)
go get github.com/go-resty/resty/v2

# JSON processing (built-in encoding/json package)
```



Study Materials

Before starting the tasks, make sure to read through the **Lab_04_Guide** which contains:

- Detailed sample code with explanations
- HTTP request/response patterns
- HTML parsing techniques
- JSON API integration examples



Task 1.1: Book Information Scraper (15 points)



Real-World Application

Every e-commerce site, from Amazon to eBay, uses web scraping to monitor competitor prices, collect product information, and track market trends. Price comparison websites like Google Shopping rely entirely on scraping data from thousands of stores. In this

task, you'll build a book information scraper - the same technology used by companies like Goodreads to build their massive book databases!

Challenge: Scrape book information from <http://books.toscrape.com/>

(<http://books.toscrape.com/>)

Instructions:

1. Create a Book struct:

```
type Book struct {
    Title      string `json:"title"`
    Price      string `json:"price"`
    Rating     string `json:"rating"`
    Availability string `json:"availability"`
    ImageURL   string `json:"image_url"`
}
```

2. **Function** scrapeBooks(url string) ([]Book, error) :

- Fetch the books page
- Parse HTML
- Extract book information from each book container
- Return slice of Book structs

3. **Function** saveBooksToJSON(books []Book, filename string) error :

- Convert books to JSON
- Save to file with pretty printing

4. **Main function:**

- Scrape first page: <http://books.toscrape.com/catalogue/page-1.html>
(<http://books.toscrape.com/catalogue/page-1.html>)
- Print summary (total books, price range)
- Save to books.json

Starter Code:

```
package main

import (
    "encoding/json"
    "fmt"
    "os"

    "golang.org/x/net/html"
)

type Book struct {
    Title      string `json:"title"`
    Price      string `json:"price"`
    Rating     string `json:"rating"`
    Availability string `json:"availability"`
    ImageURL   string `json:"image_url"`
}

func scrapeBooks(url string) ([]Book, error) {
    // TODO: Implement book scraping
    // Hint: Look for <article class="product_pod">
    // Extract: h3 > a (title), p.price_color (price),
    //          p.star-rating (rating), p.availability (stock)
    return nil, nil
}

func extractBookData(n *html.Node) Book {
    // TODO: Extract book information from article node
    // Hint: Walk through child nodes and look for specific tags/classes
    return Book{}
}

func saveBooksToJSON(books []Book, filename string) error {
    // TODO: Marshal to JSON and write to file
    return nil
}

func main() {
    url := "http://books.toscrape.com/catalogue/page-1.html"

    // TODO: Scrape books
    // TODO: Print summary
    // TODO: Save to JSON
}
```

Hints:

- Book containers have class `product_pod`
- Title is in `<h3><a title="...">`
- Price is in `<p class="price_color">`
- Rating is in `<p class="star-rating Three">` (class name contains rating)
- Use `strings.Contains()` to check class names
- Use `strings.TrimSpace()` to clean text

Expected Output:

```
Scraping books from: http://books.toscrape.com/catalogue/page-1.html
Found 20 books
```

```
Book 1:
  Title: A Light in the Attic
  Price: £51.77
  Rating: Three
  Availability: In stock
```

```
Summary:
  Total books: 20
  Average price: £45.32
```

```
Saved 20 books to books.json
```

Task 2.1: TMDB (The Movie Database) API Client (20 points)

Real-World Application

TMDB is one of the world's most popular movie databases, powering apps like Plex, Kodi, and countless streaming services. Learning to work with their API teaches you the same patterns you'll use for MangaDex in your project! Every modern app integrates with external APIs - Spotify, Google Maps, Twitter - they all work exactly the same way. Master this with movies, apply it to manga!

Challenge: Create a client for The Movie Database (TMDB) API to search and fetch movie information.

API Documentation: <https://developers.themoviedb.org/3>

(<https://developers.themoviedb.org/3>)

Free API Key: Sign up at <https://www.themoviedb.org/settings/api>

(<https://www.themoviedb.org/settings/api>) (it's free!)

Instructions:

1. Create structs for TMDB API responses:

```
type TMDBSearchResponse struct {
    Page      int `json:"page"`
    Results []struct {
        ID          int      `json:"id"`
        Title       string   `json:"title"`
        Overview    string   `json:"overview"`
        ReleaseDate string   `json:"release_date"`
        VoteAverage float64  `json:"vote_average"`
        GenreIDs    []int    `json:"genre_ids"`
        PosterPath  string   `json:"poster_path"`
    } `json:"results"`
    TotalResults int `json:"total_results"`
}

type Movie struct {
    ID          int      `json:"id"`
    Title       string   `json:"title"`
    Overview    string   `json:"overview"`
    ReleaseDate string   `json:"release_date"`
    Rating      float64  `json:"rating"`
    Genres      []string `json:"genres"`
    PosterURL   string   `json:"poster_url"`
}
```

2. **Function** searchMovies(query string, apiKey string) ([]Movie, error) :

- Make GET request to: https://api.themoviedb.org/3/search/movie?api_key={key}&query={query}

- Parse JSON response
- Convert to simplified Movie struct
- Handle errors and rate limiting

3. **Function** `getMovieDetails(id int, apiKey string) (*Movie, error) :`

- Fetch specific movie details with genres
- Endpoint: `https://api.themoviedb.org/3/movie/{id}?api_key={key}`

4. **Function** `getMovieGenres(apiKey string) (map[int]string, error) :`

- Fetch genre list from TMDB
- Endpoint: `https://api.themoviedb.org/3/genre/movie/list?api_key={key}`
- Return map of genre ID to name

5. **Function** `saveMoviesToJSON(movies []Movie, filename string) error :`

- Save movie data to JSON file

Starter Code:

```
package main

import (
    "encoding/json"
    "fmt"
    "io"
    "net/http"
    "net/url"
    "time"
)

const TMDBBaseURL = "https://api.themoviedb.org/3"

type TMDBSearchResponse struct {
    Page    int `json:"page"`
    Results []struct {
        ID      int    `json:"id"`
```

```

        Title      string `json:"title"`
        Overview    string `json:"overview"`
        ReleaseDate string `json:"release_date"`
        VoteAverage float64 `json:"vote_average"`
        GenreIDs    []int  `json:"genre_ids"`
        PosterPath  string `json:"poster_path"`
    } `json:"results"`
    TotalResults int `json:"total_results"`
}

type Movie struct {
    ID          int    `json:"id"`
    Title       string `json:"title"`
    Overview    string `json:"overview"`
    ReleaseDate string `json:"release_date"`
    Rating      float64 `json:"rating"`
    Genres      []string `json:"genres"`
    PosterURL   string `json:"poster_url"`
}

type TMDBClient struct {
    APIKey      string
    BaseURL     string
    HTTPClient  *http.Client
    GenreMap    map[int]string
}

func NewTMDBClient(apiKey string) *TMDBClient {
    return &TMDBClient{
        APIKey:  apiKey,
        BaseURL: TMDBBaseURL,
        HTTPClient: &http.Client{
            Timeout: 15 * time.Second,
        },
        GenreMap: make(map[int]string),
    }
}

func (c *TMDBClient) loadGenres() error {
    // TODO: Fetch genre list from TMDB API
    // Endpoint: /genre/movie/list
    // Store in c.GenreMap
    return nil
}

func (c *TMDBClient) searchMovies(query string) ([]Movie, error) {
    // TODO: Build URL with query parameters
    // TODO: Make HTTP GET request

```



```
// TODO: Parse JSON response
// TODO: Convert genre IDs to names using GenreMap
// TODO: Build poster URLs (base: https://image.tmdb.org/t/p/w500/)
// TODO: Return Movie structs
return nil, nil
}

func (c *TMDBClient) getMovieDetails(id int) (*Movie, error) {
    // TODO: Fetch detailed movie info including genres
    // Endpoint: /movie/{id}
    return nil, nil
}

func saveMoviesToJSON(movies []Movie, filename string) error {
    // TODO: Save to JSON file with pretty printing
    return nil
}

func main() {
    // Get API key from environment or hardcode for testing
    apiKey := "YOUR_API_KEY_HERE" // Replace with your TMDB API key

    client := NewTMDBClient(apiKey)

    // Load genres first
    fmt.Println("Loading movie genres...")
    err := client.loadGenres()
    if err != nil {
        fmt.Printf("Error loading genres: %v\n", err)
        return
    }

    // Search for movies
    query := "inception"
    fmt.Printf("Searching for: %s\n", query)

    // TODO: Search movies
    // TODO: Display results
    // TODO: Save to JSON
}
```

Hints:

- TMDB API requires an API key in every request
- Use `url.QueryEscape()` for query parameters

- Genre IDs need to be converted to names using the genre list endpoint
- Poster path is relative - combine with base URL:
`https://image.tmdb.org/t/p/w500/ + poster_path`
- Add proper error handling for network failures
- TMDB has rate limiting: 40 requests per 10 seconds

Expected Output:

```
Loading movie genres...
```

```
Loaded 19 genres
```

```
Searching for: inception
```

```
Found 20 movies
```

```
Movie 1:
```

```
  ID: 27205
```

```
  Title: Inception
```

```
  Release Date: 2010-07-15
```

```
  Rating: 8.4/10
```

```
  Genres: [Action, Science Fiction, Adventure]
```

```
  Overview: Cobb, a skilled thief who commits corporate espionage...
```

```
Movie 2:
```

```
  ID: 198663
```

```
  Title: The Inception of Doom
```

```
  Release Date: 2016-05-24
```

```
  Rating: 5.2/10
```

```
  Genres: [Action, Horror]
```

```
  ...
```

```
Saved 20 movies to tmdb_results.json
```

Note: This is identical to how you'll work with MangaDex API in your project! The patterns are the same:

- API authentication (API key vs JWT)
- JSON response parsing
- Data transformation
- Error handling

- Rate limiting

Task 2.2: Multi-Source Movie Aggregator (20 points)

Real-World Application

This is exactly the pattern you'll use in MangaHub! Professional data aggregation platforms like Metacritic, Rotten Tomatoes, and Google Movies collect information from multiple sources to provide comprehensive data. You'll build a mini version that combines data from web scraping and API calls - the same architecture you'll need for your manga project. Learn it with movies, apply it to manga!

Challenge: Combine web scraping and API calls to create a comprehensive movie database.

Instructions:

1. Create a unified `MovieInfo` struct:

```
type MovieInfo struct {
    ID           string `json:"id"`
    Title        string `json:"title"`
    Director     string `json:"director,omitempty"`
    Year         int    `json:"year"`
    Description   string `json:"description"`
    Genres       []string `json:"genres"`
    Rating       float64 `json:"rating"`
    Duration     int    `json:"duration_minutes,omitempty"`
    Source       string `json:"source" // "tmdb", "imdb", "scraped", etc.`
    LastUpdated  string `json:"last_updated"`
}
```

2. **Interface** `MovieSource` :

```
type MovieSource interface {  
    GetMovies(query string, limit int) ([]MovieInfo, error)  
    GetName() string  
}
```

3. Implement sources:

- TMDBSource - API client from Task 2.1
- MockScraperSource - Mock data or simple scraper

4. **Function** aggregateMovies(sources []MovieSource, query string) ([]MovieInfo, error) :

- Query all sources concurrently using goroutines
- Combine results
- Remove duplicates (by title similarity)
- Merge data from same movie (keep highest rating, combine genres)
- Sort by rating/relevance

5. **Function** calculateSimilarity(title1, title2 string) float64 :

- Calculate title similarity (simple comparison or Levenshtein distance)
- Return score 0.0 to 1.0

6. **Function** generateReport(movies []MovieInfo) :

- Create summary statistics
- Group by source
- Count by genre
- Average ratings
- Format as readable report

Starter Code:

```

package main

import (
    "encoding/json"
    "fmt"
    "strings"
    "sync"
    "time"
)

type MovieInfo struct {
    ID          string    `json:"id"`
    Title       string    `json:"title"`
    Director    string    `json:"director,omitempty"`
    Year        int       `json:"year"`
    Description  string    `json:"description"`
    Genres      []string  `json:"genres"`
    Rating      float64   `json:"rating"`
    Duration    int       `json:"duration_minutes,omitempty"`
    Source      string    `json:"source"`
    LastUpdated string    `json:"last_updated"`
}

type MovieSource interface {
    GetMovies(query string, limit int) ([]MovieInfo, error)
    GetName() string
}

// TMDBSource - uses TMDB API
type TMDBSource struct {
    client *TMDBClient
}

func NewTMDBSource(apiKey string) *TMDBSource {
    return &TMDBSource{
        client: NewTMDBClient(apiKey),
    }
}

func (t *TMDBSource) GetMovies(query string, limit int) ([]MovieInfo, error) {
    // TODO: Use TMDB client to search movies
    // TODO: Convert Movie structs to MovieInfo
    // TODO: Set source to "TMDB"
    return nil, nil
}

func (t *TMDBSource) GetName() string {

```

```

        return "TMDB"
    }

// MockScraperSource - simulates a scraped source
type MockScraperSource struct {
    name string
}

func NewMockScraperSource(name string) *MockScraperSource {
    return &MockScraperSource{name: name}
}

func (m *MockScraperSource) GetMovies(query string, limit int) ([]MovieInfo,
// TODO: Return mock data or implement real scraper
// For practice, return some mock movies
return []MovieInfo{
    {
        ID:          "mock-1",
        Title:       "The " + strings.Title(query),
        Year:        2020,
        Description: "A great movie about " + query,
        Genres:      []string{"Drama", "Action"},
        Rating:      7.5,
        Source:      m.name,
        LastUpdated: time.Now().Format(time.RFC3339),
    },
}, nil
}

func (m *MockScraperSource) GetName() string {
    return m.name
}

// MovieAggregator - combines multiple sources
type MovieAggregator struct {
    Sources []MovieSource
}

func NewMovieAggregator(sources ...MovieSource) *MovieAggregator {
    return &MovieAggregator{Sources: sources}
}

func (a *MovieAggregator) Search(query string, limitPerSource int) ([]MovieInfo,
var allMovies []MovieInfo
var mu sync.Mutex
var wg sync.WaitGroup

// TODO: Query all sources concurrently using goroutines

```

```

// TODO: Combine results
// TODO: Remove duplicates
// TODO: Merge duplicate movie data

for _, source := range a.Sources {
    wg.Add(1)
    go func(src MovieSource) {
        defer wg.Done()

        // Query source
        movies, err := src.GetMovies(query, limitPerSource)
        if err != nil {
            fmt.Printf("Error from %s: %v\n", src.GetName(), err)
            return
        }

        // Add to results
        mu.Lock()
        allMovies = append(allMovies, movies...)
        mu.Unlock()
    }(source)
}

wg.Wait()

// Deduplicate and merge
deduplicated := deduplicateMovies(allMovies)

return deduplicated, nil
}

func deduplicateMovies(movies []MovieInfo) []MovieInfo {
    // TODO: Find duplicate movies by title similarity
    // TODO: Merge data from duplicates (keep best rating, combine genres)
    // TODO: Return unique movies

    // Simple implementation - remove exact title matches
    seen := make(map[string]bool)
    var unique []MovieInfo

    for _, movie := range movies {
        titleKey := strings.ToLower(strings.TrimSpace(movie.Title))
        if !seen[titleKey] {
            seen[titleKey] = true
            unique = append(unique, movie)
        }
    }
}

```

```

        return unique
    }

func calculateSimilarity(title1, title2 string) float64 {
    // TODO: Implement similarity calculation
    // Simple version: case-insensitive exact match
    t1 := strings.ToLower(strings.TrimSpace(title1))
    t2 := strings.ToLower(strings.TrimSpace(title2))

    if t1 == t2 {
        return 1.0
    }

    // Check if one contains the other
    if strings.Contains(t1, t2) || strings.Contains(t2, t1) {
        return 0.8
    }

    return 0.0
}

func generateReport(movies []MovieInfo) {
    fmt.Println("\n=== Movie Aggregation Report ===")
    fmt.Printf("Total movies found: %d\n\n", len(movies))

    // Count by source
    sourceCount := make(map[string]int)
    for _, movie := range movies {
        sourceCount[movie.Source]++
    }

    fmt.Println("Movies by Source:")
    for source, count := range sourceCount {
        fmt.Printf("  - %s: %d movies\n", source, count)
    }

    // Count by genre
    genreCount := make(map[string]int)
    for _, movie := range movies {
        for _, genre := range movie.Genres {
            genreCount[genre]++
        }
    }

    fmt.Println("\nTop Genres:")
    // TODO: Sort and display top genres

    // Calculate average rating

```



```
var totalRating float64
ratedCount := 0
for _, movie := range movies {
    if movie.Rating > 0 {
        totalRating += movie.Rating
        ratedCount++
    }
}

if ratedCount > 0 {
    avgRating := totalRating / float64(ratedCount)
    fmt.Printf("\nAverage Rating: %.2f/10\n", avgRating)
}
}

func saveToJSON(movies []MovieInfo, filename string) error {
    // TODO: Save to JSON file
    data, err := json.MarshalIndent(movies, "", " ")
    if err != nil {
        return err
    }

    // TODO: Write to file
    fmt.Printf("\nSaved %d movies to %s\n", len(movies), filename)
    return nil
}

func main() {
    apiKey := "YOUR_TMDB_API_KEY" // Replace with your key

    // Create aggregator with multiple sources
    aggregator := NewMovieAggregator(
        NewTMDBSource(apiKey),
        NewMockScraperSource("MovieScraper"),
    )

    query := "spider-man"
    fmt.Printf("=== Multi-Source Movie Aggregator ===\n")
    fmt.Printf("Searching for: %s\n\n", query)

    // Search all sources
    movies, err := aggregator.Search(query, 10)
    if err != nil {
        fmt.Printf("Error: %v\n", err)
        return
    }

    // Display results
```

```
fmt.Printf("\nFound %d movies after deduplication\n", len(movies))

// Show first 3
for i, movie := range movies {
    if i < 3 {
        fmt.Printf("\n%d. %s (%d)\n", i+1, movie.Title, movie.Rating)
        fmt.Printf("    Source: %s\n", movie.Source)
        fmt.Printf("    Rating: %.1f/10\n", movie.Rating)
        fmt.Printf("    Genres: %v\n", movie.Genres)
    }
}

// Generate report
generateReport(movies)

// Save to JSON
saveToJSON(movies, "aggregated_movies.json")
}
```

Expected Output:

=== Multi-Source Movie Aggregator ===

Searching for: spider-man

Querying TMDB... Found 15 results

Querying MovieScraper... Found 1 results

Found 16 movies after deduplication

1. Spider-Man (2002)
Source: TMDB
Rating: 7.3/10
Genres: [Action, Fantasy]
2. Spider-Man: No Way Home (2021)
Source: TMDB
Rating: 8.1/10
Genres: [Action, Adventure, Science Fiction]
3. The Amazing Spider-Man (2012)
Source: TMDB
Rating: 6.9/10
Genres: [Action, Adventure, Fantasy]

=== Movie Aggregation Report ===

Total movies found: 16

Movies by Source:

- TMDB: 15 movies
- MovieScraper: 1 movies

Top Genres:

- Action: 16
- Adventure: 12
- Science Fiction: 8
- Fantasy: 5

Average Rating: 7.4/10

Saved 16 movies to aggregated_movies.json

Key Learning Points:

- **Concurrent data fetching** - using goroutines (you'll do this for manga APIs)
- **Data normalization** - converting different formats to unified structure

- **Deduplication** - handling same item from multiple sources
- **Error resilience** - continue if one source fails
- **Data merging** - combine information from multiple sources

Connection to MangaHub:

This exact pattern applies to your project:

- Replace MovieSource with MangaSource
- Replace TMDBSource with MangaDexSource
- Add more sources: MyAnimeListSource , ScrapedMangaSource
- Use same aggregation logic!

Task 3.1: Paginated Content Scraper (15 points)

Real-World Application

Most websites don't show all content on one page. Amazon product listings, news sites, social media feeds - they all use pagination. Job boards like Indeed and LinkedIn scrape thousands of job postings by navigating through pages. E-commerce price trackers monitor entire product catalogs. Mastering pagination is essential for collecting large datasets!

Challenge: Scrape multiple pages of content and handle pagination.

Instructions:

1. **Function** `scrapePaginatedBooks(baseUrl string, maxPages int) ([]Book, *ScraperStats, error) :`
 - Start from page 1
 - Extract books from each page
 - Follow pagination links
 - Stop at maxPages or when no more pages

- Add rate limiting (delay between requests)

2. **Function** `getNextPageURL(doc *html.Node, currentURL string) (string, bool) :`

- Find "next page" link
- Resolve relative URLs to absolute
- Return URL and whether next page exists

3. **Features to implement:**

- Progress indicator (show current page being scraped)
- Error recovery (retry failed pages)
- Rate limiting (1 request per second)
- Statistics tracking (total books, pages scraped, errors)

Starter Code:

```
package main

import (
    "fmt"
    "time"
)

type ScraperStats struct {
    PagesScraped int
    BooksFound   int
    Errors       int
    StartTime    time.Time
    EndTime      time.Time
}

func scrapePaginatedBooks(baseUrl string, maxPages int) ([]Book, *ScraperStats) {
    stats := &ScraperStats{
        StartTime: time.Now(),
    }

    var allBooks []Book
```

```

currentURL := baseURL

for page := 1; page <= maxPages; page++ {
    // TODO: Print progress
    // TODO: Scrape current page
    // TODO: Add rate limiting (time.Sleep)
    // TODO: Find next page URL
    // TODO: Update statistics

    // If no next page, break

    // Rate limit: 1 request per second
    time.Sleep(1 * time.Second)
}

stats.EndTime = time.Now()
return allBooks, stats, nil
}

func getNextPageURL(doc *html.Node, baseURL string) (string, bool) {
    // TODO: Find "next" button/link
    // TODO: Extract href
    // TODO: Resolve relative to absolute URL
    return "", false
}

func printStats(stats *ScraperStats) {
    // TODO: Print scraping statistics
}

func main() {
    baseURL := "http://books.toscrape.com/catalogue/page-1.html"
    maxPages := 5

    fmt.Printf("Starting paginated scraper...\n")
    fmt.Printf("Max pages: %d\n\n", maxPages)

    // TODO: Scrape multiple pages
    // TODO: Print statistics
    // TODO: Save all books to JSON
}

```

Expected Output:

```
Starting paginated scraper...
```

```
Max pages: 5
```

```
Scraping page 1/5... Found 20 books
```

```
Scraping page 2/5... Found 20 books
```

```
Scraping page 3/5... Found 20 books
```

```
Scraping page 4/5... Found 20 books
```

```
Scraping page 5/5... Found 20 books
```

```
=== Scraping Statistics ===
```

```
Pages scraped: 5
```

```
Total books found: 100
```

```
Errors: 0
```

```
Duration: 5.2 seconds
```

```
Average books per page: 20.0
```

```
Saved 100 books to paginated_books.json
```

Task 4.1: Movie Database Builder (15 points) - REQUIRED

Real-World Application

This is your practice run for the MangaHub project! You'll build the exact data collection pipeline pattern you'll need, but with movies instead of manga. Companies like IMDB, Letterboxd, and JustWatch started with databases built from API and scraped data. You're learning the architecture you'll replicate for your manga tracking system!

Challenge: Build a complete movie database collection and management system that demonstrates the same patterns you'll use for MangaHub.

Instructions:

Prerequisites: This task builds upon:

- Task 2.1: TMDBClient struct and methods
- Task 2.2: MovieInfo struct and MovieSource interface

Make sure you have completed those tasks first, or copy the necessary code.

1. Create a complete data pipeline:

- Collect from TMDB API
- Process and normalize data
- Store in structured JSON
- Provide query interface

2. Database Structure:

```
type MovieDatabase struct {
    Movies      map[string]MovieInfo `json:"movies"`           // key: movie ID
    Genres       map[string][]string   `json:"genres"`           // key: genre name
    Directors    map[string][]string   `json:"directors"`        // key: director name
    Years        map[int][]string      `json:"years"`            // key: year, value: movies
    LastUpdated  time.Time             `json:"last_updated"`
    TotalCount   int                   `json:"total_count"`
}
```

3. Implement full CRUD operations:

```
type MovieDB interface {
    Add(movie MovieInfo) error
    Get(id string) (*MovieInfo, error)
    Search(query string) ([]MovieInfo, error)
    GetByGenre(genre string) ([]MovieInfo, error)
    GetByYear(year int) ([]MovieInfo, error)
    GetByDirector(director string) ([]MovieInfo, error)
    Update(movie MovieInfo) error
    Delete(id string) error
    Save(filename string) error
    Load(filename string) error
}
```

4. Build collection pipeline:

- Search for popular keywords: "action", "comedy", "drama", "horror", "sci-fi", etc.

- Collect at least 10 movies per category
- Total target: 100+ movie entries
- Store all data in `movie_database.json`

5. Statistics and reporting:

- Total movies by genre
- Movies by year/decade
- Rating distribution
- Most common directors

Starter Code:

```
package main

import (
    "encoding/json"
    "fmt"
    "os"
    "strings"
    "time"
)

type MovieDatabase struct {
    Movies      map[string]MovieInfo `json:"movies"`
    Genres      map[string][]string  `json:"genres"`
    Directors   map[string][]string  `json:"directors"`
    Years       map[int][]string      `json:"years"`
    LastUpdated time.Time             `json:"last_updated"`
    TotalCount  int                   `json:"total_count"`
}

func NewMovieDatabase() *MovieDatabase {
    return &MovieDatabase{
        Movies:    make(map[string]MovieInfo),
        Genres:    make(map[string][]string),
        Directors: make(map[string][]string),
        Years:     make(map[int][]string),
    }
}
```

```

func (db *MovieDatabase) Add(movie MovieInfo) error {
    // TODO: Check if movie already exists
    movieID := movie.ID

    // TODO: Add movie to Movies map
    db.Movies[movieID] = movie

    // TODO: Update genre index
    for _, genre := range movie.Genres {
        // Add movie ID to genre's list
        db.Genres[genre] = append(db.Genres[genre], movieID)
    }

    // TODO: Update director index (if director exists)
    if movie.Director != "" {
        db.Directors[movie.Director] = append(db.Directors[movie.Dire
    }

    // TODO: Update year index
    if movie.Year > 0 {
        db.Years[movie.Year] = append(db.Years[movie.Year], movieID)
    }

    // TODO: Update count
    db.TotalCount++

    return nil
}

func (db *MovieDatabase) Get(id string) (*MovieInfo, error) {
    // TODO: Retrieve movie by ID
    movie, exists := db.Movies[id]
    if !exists {
        return nil, fmt.Errorf("movie not found: %s", id)
    }
    return &movie, nil
}

func (db *MovieDatabase) Search(query string) ([]MovieInfo, error) {
    // TODO: Search movies by title (case-insensitive)
    var results []MovieInfo
    query = strings.ToLower(query)

    for _, movie := range db.Movies {
        if strings.Contains(strings.ToLower(movie.Title), query) {
            results = append(results, movie)
        }
    }
}

```

```
        return results, nil
    }

func (db *MovieDatabase) GetByGenre(genre string) ([]MovieInfo, error) {
    // TODO: Get all movies in genre using genre index
    var results []MovieInfo

    movieIDs, exists := db.Genres[genre]
    if !exists {
        return results, nil
    }

    for _, id := range movieIDs {
        if movie, err := db.Get(id); err == nil {
            results = append(results, *movie)
        }
    }

    return results, nil
}

func (db *MovieDatabase) GetByYear(year int) ([]MovieInfo, error) {
    // TODO: Get all movies from specific year
    var results []MovieInfo

    movieIDs, exists := db.Years[year]
    if !exists {
        return results, nil
    }

    for _, id := range movieIDs {
        if movie, err := db.Get(id); err == nil {
            results = append(results, *movie)
        }
    }

    return results, nil
}

func (db *MovieDatabase) GetByDirector(director string) ([]MovieInfo, error) {
    // TODO: Get all movies by director
    var results []MovieInfo

    movieIDs, exists := db.Directors[director]
    if !exists {
        return results, nil
    }
}
```

```

    for _, id := range movieIDs {
        if movie, err := db.Get(id); err == nil {
            results = append(results, *movie)
        }
    }

    return results, nil
}

func (db *MovieDatabase) Save(filename string) error {
    // TODO: Marshal database to JSON
    // TODO: Write to file with pretty printing
    data, err := json.MarshalIndent(db, "", " ")
    if err != nil {
        return err
    }

    return os.WriteFile(filename, data, 0644)
}

func (db *MovieDatabase) Load(filename string) error {
    // TODO: Read file
    // TODO: Unmarshal JSON to database
    data, err := os.ReadFile(filename)
    if err != nil {
        return err
    }

    return json.Unmarshal(data, db)
}

func (db *MovieDatabase) PrintStatistics() {
    fmt.Println("\n=== Movie Database Statistics ===")
    fmt.Printf("Total Movies: %d\n", db.TotalCount)
    fmt.Printf("Total Genres: %d\n", len(db.Genres))
    fmt.Printf("Total Directors: %d\n", len(db.Directors))
    fmt.Printf("Year Range: %d entries\n\n", len(db.Years))

    // Genre statistics
    fmt.Println("Movies by Genre:")
    for genre, movieIDs := range db.Genres {
        if len(movieIDs) > 0 {
            fmt.Printf("  - %s: %d movies\n", genre, len(movieIDs))
        }
    }

    // TODO: Add more statistics

```

```

    // - Rating distribution
    // - Decade distribution
    // - Top directors
}

// Collection pipeline
func buildMovieDatabase(apiKey string) (*MovieDatabase, error) {
    db := NewMovieDatabase()
    client := NewTMDBClient(apiKey)

    // Load genres first
    fmt.Println("Loading movie genres from TMDB...")
    err := client.loadGenres()
    if err != nil {
        return nil, fmt.Errorf("failed to load genres: %w", err)
    }

    // Define search queries to build diverse database
    searchQueries := []string{
        "marvel", "star wars", "harry potter", "batman",
        "comedy", "horror", "romance", "action",
        "2023", "2022", "2021", "classic",
    }

    fmt.Println("\nBuilding movie database...")

    for i, query := range searchQueries {
        fmt.Printf("[%d/%d] Searching for: %s\n", i+1, len(searchQuer

        // TODO: Search movies using TMDB client
        movies, err := client.searchMovies(query)
        if err != nil {
            fmt.Printf("  Error: %v\n", err)
            continue
        }

        // Add to database
        added := 0
        for _, movie := range movies {
            movieInfo := MovieInfo{
                ID:          fmt.Sprintf("%d", movie.ID),
                Title:        movie.Title,
                Year:         extractYear(movie.ReleaseDate),
                Description:  movie.Overview,
                Genres:       movie.Genres,
                Rating:      movie.Rating,
                Source:      "TMDB",
                LastUpdated:  time.Now().Format(time.RFC3339),
            }
        }
    }
}

```

```

    }

    // Only add if not duplicate
    if _, exists := db.Movies[movieInfo.ID]; !exists {
        db.Add(movieInfo)
        added++
    }
}

fmt.Printf(" Added %d new movies (found %d total)\n", added,

// Rate limiting - be nice to the API
time.Sleep(1 * time.Second)
}

db.LastUpdated = time.Now()
fmt.Printf("\nDatabase building complete!\n")

return db, nil
}

func extractYear(dateStr string) int {
    // TODO: Extract year from "YYYY-MM-DD" format
    if len(dateStr) >= 4 {
        var year int
        fmt.Sscanf(dateStr[:4], "%d", &year)
        return year
    }
    return 0
}

func main() {
    fmt.Println("=== Movie Database Builder ===\n")

    apiKey := "YOUR_TMDB_API_KEY" // Replace with your key

    // Build database
    fmt.Println("Starting database collection...")
    db, err := buildMovieDatabase(apiKey)
    if err != nil {
        fmt.Printf("Error building database: %v\n", err)
        return
    }

    // Print statistics
    db.PrintStatistics()

    // Test search

```

```

fmt.Println("\n=== Testing Search Functions ===")

// Search by title
fmt.Println("\nSearching for 'spider':")
results, _ := db.Search("spider")
for i, movie := range results {
    if i < 3 {
        fmt.Printf("  %d. %s (%d) - Rating: %.1f\n",
            i+1, movie.Title, movie.Year, movie.Rating)
    }
}

// Search by genre
if len(db.Genres) > 0 {
    var firstGenre string
    for genre := range db.Genres {
        firstGenre = genre
        break
    }

    fmt.Printf("\nMovies in genre '%s':\n", firstGenre)
    genreMovies, _ := db.GetByGenre(firstGenre)
    for i, movie := range genreMovies {
        if i < 3 {
            fmt.Printf("  %d. %s (%d)\n", i+1, movie.Title
        }
    }
}

// Save to file
filename := "movie_database.json"
err = db.Save(filename)
if err != nil {
    fmt.Printf("Error saving database: %v\n", err)
    return
}

// Get file size
fileInfo, _ := os.Stat(filename)
sizeKB := fileInfo.Size() / 1024

fmt.Printf("\n✓ Database saved successfully!\n")
fmt.Printf("  File: %s\n", filename)
fmt.Printf("  Size: %d KB\n", sizeKB)
fmt.Printf("  Movies: %d\n", db.TotalCount)
}

```

Note: This code requires the `TMDBClient` from Task 2.1 and `MovieInfo` struct from Task 2.2 to work.

Expected Output:

```
=== Movie Database Builder ===

Starting database collection...
Loading movie genres from TMDB...
Loaded 19 genres

Building movie database...
[1/12] Searching for: marvel
  Added 15 new movies (found 20 total)
[2/12] Searching for: star wars
  Added 12 new movies (found 20 total)
...

=== Movie Database Statistics ===
Total Movies: 147
Total Genres: 15
Total Directors: 0
Year Range: 42 entries

Movies by Genre:
- Action: 68 movies
- Adventure: 52 movies
- Science Fiction: 41 movies
...

✓ Database saved successfully!
File: movie_database.json
Size: 234 KB
Movies: 147
```

Submission Requirements (100 points)

Tasks (85 points)

- ☐ Task 1.1: Book Information Scraper (15 points)
- ☐ Task 2.1: TMDB API Client (20 points)

- ☐ Task 2.2: Multi-Source Movie Aggregator (20 points)
- ☐ Task 3.1: Paginated Content Scraper (15 points)
- ☐ Task 4.1: Movie Database Builder (15 points) - **REQUIRED for Final Project**

Code Quality (15 points)

- ☐ Code compiles and runs (5 points)
- ☐ Proper error handling (4 points)
- ☐ Clean, readable code with comments (3 points)
- ☐ Data properly saved to JSON (3 points)

Bonus (+10 points)

- ☐ Implement concurrent scraping (use goroutines) (+5 points)
- ☐ Add image download functionality (+3 points)
- ☐ Implement retry mechanism with exponential backoff (+2 points)

Testing Checklist

Before submitting, verify:

1. ✓ All scrapers successfully fetch data
2. ✓ API client handles errors gracefully
3. ✓ JSON files are properly formatted
4. ✓ Rate limiting is implemented (no API abuse)
5. ✓ Pagination works correctly
6. ✓ Data is deduplicated properly
7. ✓ Database can be saved and loaded

Connection to Final Project (MangaHub)

This lab directly prepares you for the MangaHub project by teaching transferable patterns:

What You Practiced (Movies):

1. **Task 2.1:** TMDB API integration → **Apply to:** MangaDex API
2. **Task 2.2:** Multi-source aggregation → **Apply to:** Manga from multiple APIs
3. **Task 4.1:** Movie database builder → **Apply to:** Manga database

Direct Skill Transfer:

Lab Skill (Movies)	Project Application (Manga)
TMDB API Client	MangaDex API Client
Movie genres	Manga genres
Rating system	Manga rating/popularity
Multi-source aggregation	Combine MangaDex + MyAnimeList
JSON database structure	Manga database structure
Search & filter operations	Manga search & discovery
Rate limiting patterns	API rate limit handling

Code Patterns You Can Reuse:

```
// Lab Pattern (Movies)
type TMDBClient struct {
    APIKey    string
    BaseURL   string
    HTTPClient *http.Client
}

// Project Pattern (Manga) - Same structure!
type MangaDexClient struct {
    BaseURL    string
    HTTPClient *http.Client
    AuthToken  string // Instead of API key
}
```

What Changes for MangaHub:

1. **API Endpoint:** `api.themoviedb.org` → `api.mangadex.org`
2. **Data Fields:** Director → Author, Runtime → Chapters
3. **Search Queries:** Movie titles → Manga titles
4. **Everything Else:** Same patterns, same architecture!

Next Steps for Your Project:

1.  Complete all lab tasks with movies
2.  Replace movie structs with manga structs
3.  Replace TMDB client with MangaDex client
4.  Add more manga sources (MyAnimeList, AniList)
5.  Add manga-specific features (chapters, reading status)

You're now ready to build the data collection layer of MangaHub!

Getting Help

During Lab:

-  Ask AI: "How do I parse HTML in Go?" or "TMDB API example"

-  Ask classmates
-  Ask instructor
-  Check the Sample Guide (Lab_07_Sample_Guide.md)

Useful Prompts for AI:

- "Go HTML parsing example"
- "HTTP GET request with JSON response in Go"
- "How to handle pagination in web scraping"
- "TMDB API search movies in Go"
- "Build URL with query parameters in Go"

Resources:

- TMDB API Docs: <https://developers.themoviedb.org/3> (<https://developers.themoviedb.org/3>)
- Go HTML package: <https://pkg.go.dev/golang.org/x/net/html>
(<https://pkg.go.dev/golang.org/x/net/html>)
- HTTP client: <https://pkg.go.dev/net/http> (<https://pkg.go.dev/net/http>)
- JSON encoding: <https://pkg.go.dev/encoding/json> (<https://pkg.go.dev/encoding/json>)

End of Lab 04 Tasks